

Doubly Linked List – Search For Element –Space Complexity and Algorithm

Program :

```
void searchForElement(int data)
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    int pos = 1;
    while (temp != nullptr)
    {
        if (temp->data == data)
        {
            cout << "Element " << data << " found at
position " << pos << "." << endl;
            return;
        }
        temp = temp->next;
        pos++;
    }
    cout << "Element " << data << " not found in the list."
<< endl;
}
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the searchForElement function, the new variables created are:

Node *temp: A single temporary pointer to traverse the list. The size of a pointer is constant.

int pos: An integer variable to keep track of the current position. The size of an integer is also constant.

The function doesn't allocate any new nodes. Since the amount of extra memory used is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void searchForElement(int data)`.

The input is a single integer, data, which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Search for an Element (Doubly Linked List)

DSEARCH(START, ITEM)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty." and EXIT.

[End of If structure.]

2. [INITIALIZE POINTERS FOR TRAVERSAL]

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Set POS := 1 [POS is an integer to track the node position].

3. [TRAVERSE THE LIST TO FIND THE ITEM]

Repeat while PTR $\neq$ NULL:

a. If INFO[PTR] = ITEM, then: [Check if the current node contains the item].

Write: "Element ", ITEM, " found at position ", POS.

EXIT. [End the search as the item is found].

[End of If structure]

b. Set PTR := NEXTLINK[PTR] [Move to the next node].

c. Set POS := POS + 1 [Increment the position counter].

[END of Loop]

4. [REPORT IF ITEM WAS NOT FOUND]

Write: "Element ", ITEM, " not found in the list." [This line is only reached if the loop finishes].

5. Exit
